

COMS / SE 4170 Introduction to Software Testing Spring 2026

Tuesday and Thursday 9:30-10:45 AM
Location: Carver 0205

1

Why This Class?



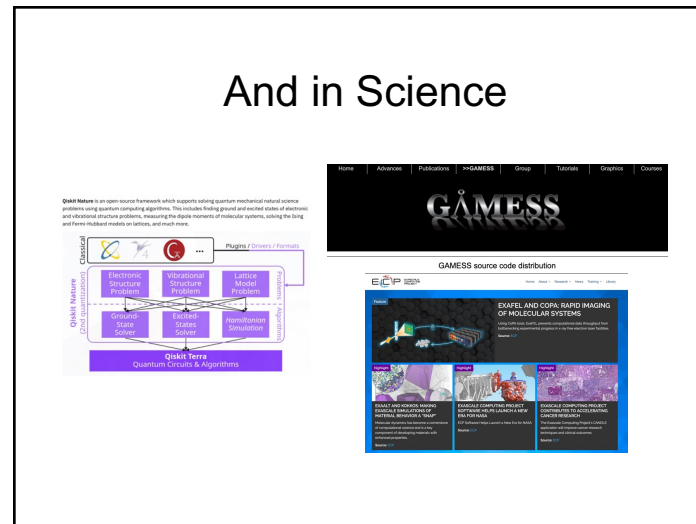
2

Software is Everywhere



3

And in Science



4

And Sometimes it Fails

```
*** CHECK YOUR INPUT CHARGE AND MULTIPLICITY ***
THERE ARE 27 ELECTRONS, WITH CHARGE ICHARG= 0
BUT YOU SELECTED MULTIPLICITY MULT= 1
EXECUTION OF GAMESS TERMINATED -ABNORMALLY- AT 18:08:36 16-SEP-2025
580000 WORDS OF DYNAMIC MEMORY USED
STEP CPU TIME = 0.01 TOTAL CPU TIME = 0.0 ( 0.0 MIN)
TOTAL WALL CLOCK TIME= 0.1 SECONDS, CPU UTILIZATION IS 5.33%
DDI Process 0: error code 911
ddkick.x: application process 0 quit unexpectedly.
ddkick.x: Fatal error detected.
The error is most likely to be in the application, so check for
input errors, disk space, memory needs, application bugs, etc.
ddkick.x will now clean up all processes, and exit...
ddkick.x: Sending kill signal to DDI processes.
ddkick.x: Execution terminated due to error(s).
inset echo
```

5

High Profile

The Heartbleed Bug

The Heartbleed Bug is a serious vulnerability in the popular OpenSSL cryptographic software library. This weakness allows stealing the information protected, under normal conditions, by the SSL/TLS encryption used to secure the Internet. SSL/TLS provides communication security and privacy over the Internet for applications such as web, email, instant messaging (IM) and some virtual private networks (VPNs).

The Heartbleed bug allows anyone on the Internet to read the systems protected by the vulnerable versions of the OpenSSL software. This compromises the secret keys used to identify providers and to encrypt the traffic, the names and password users and the actual content. This allows attackers to eavesdrop on communications, steal data directly from the services and use impersonate services and users.



■ THERAC-25 radiation machine : Poor testing of safety-critical software can cost lives : 3 patients were killed

■ NASA's Mars Lander: September 1999, crashed due to a units integration fault



6

Toyota Acceleration

Toyota Case: Single Bit Flip That Killed

Junko Yoshida
10/25/2010 03:35 PM EDT

During the trial, embedded systems experts who reviewed Toyota's electronic throttle source code testified that they found Toyota's source code defective, and that it contains bugs -- including bugs that can cause unintended acceleration.

"We did a few things that NASA apparently did not have time to do," Barr said. For one thing, by looking within the real-time operating system, the experts identified "unprotected critical variables." They obtained and reviewed the source code for the "sub-CPU," and they uncovered gaps and defects in the throttle fail safes.

The experts demonstrated that "the defects we found were linked to unintended acceleration through vehicle testing," Barr said. "We also obtained and reviewed the source code for the black box and found that it can record false information about the driver's actions in the final seconds before a crash."

Stack overflow and software bugs led to memory corruption, he said. And it turns out that the crux of the issue was these memory corruptions, which acted "like ricocheting bullets."

Barr also said more than half the dozens of tasks' deaths studied by the experts in their experiments were not detected by any fail safe."

© Copyright 2014, Philip Koopman. CC Attribution 4.0 International license.

Bookout Trial Reporting

http://www.eetimes.com/document.asp?doc_id=1319903&page_number=1
(excerpts)

"Task X death in combination with other task deaths"

14

7

Airport Problems (July 2024)

2024 CrowdStrike-related IT outages

Article Talk

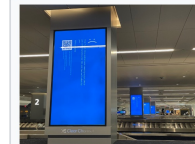
Read Edit View history Tools

From Wikipedia, the free encyclopedia

On 19 July 2024, the American cybersecurity company CrowdStrike distributed a faulty update to its Falcon Sensor security software that caused widespread problems with Microsoft Windows computers running the software. As a result, roughly 8.5 million systems crashed and were unable to properly restart^[1] in what has been called the largest outage in the history of information technology^[2] and "historic in scale".^[3]

The outage disrupted daily life, businesses, and governments around the world. Many industries were affected--airlines, airports, banks, hotels, hospitals, manufacturing, stock markets, broadcasting, gas stations, retail stores, and governmental services, such as emergency services and websites.^{[4][5]} The worldwide financial

2024 CrowdStrike-related IT outages



Multiple blue screens of death caused by a faulty software update on baggage carousels at LaGuardia Airport, New York City

Date 19 July 2024; 18 months ago

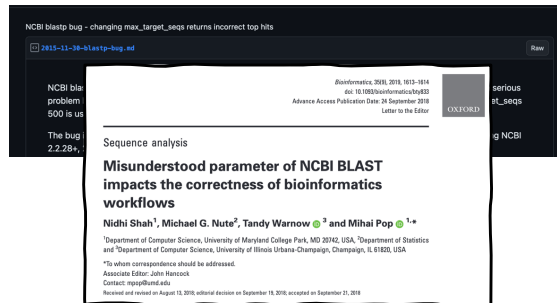
Location Worldwide

Type IT outage, computer crash

Also see CrowdStrike software update

8

Subtle Faults are Easy to Miss



9

Consequences



National Institutes of Standards (NIST)– up to \$59 Billion per year – could be cut in ½ with better testing



Government accountability report (GAO) report DOD loses \$8 Billion annually due to rework [2004]

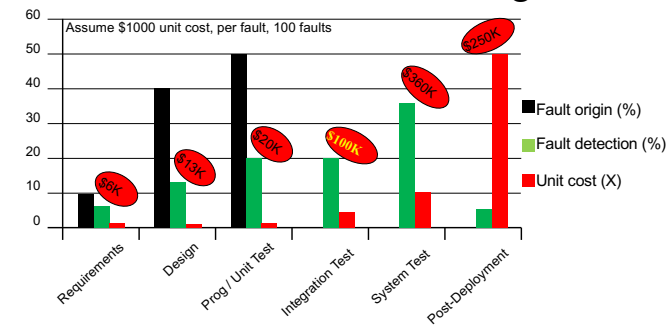
10

Consequence Cont.

- Huge losses due to web application failures
 - Financial services : \$6.5 million per hour (just in USA!)
 - Credit card sales applications : \$2.4 million per hour (in USA)
 - 2007 : Symantec says that most **security vulnerabilities** are due to faulty software

11

Cost of Late Testing



Software Engineering Institute; Carnegie Mellon University; Handbook CMU/SEI-96-HB-002

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

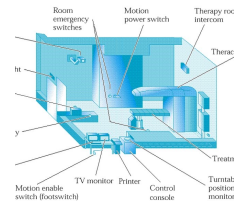
12

12

And

- This does not consider loss of lives...

THERAC-25



13

Need for Testing Isn't New



“an analyzing process must equally have been performed in order to furnish the Analytical Engine with the necessary operative data; and that herein may also lie a possible source of error. Granted that the actual mechanism is unerring in its processes, the cards may give it wrong orders.” – Ada, Countess Lovelace (notes on Babbage's Analytical Engine)



“It has been just so in all of my inventions. The first step is an intuition, and comes with a burst, then difficulties arise—this thing gives out and /it is/ then that 'Bugs'—as such little faults and difficulties are called—show themselves and months of intense watching, study and labor are requisite. . .” – Thomas Edison

14

Objectives – This Course



Learn to design tests to meet realistic coverage criteria



Be in a position, to apply standard software testing techniques



Gain exposure to some testing tools



Gain exposure to at least one advanced testing method in more depth

15

Pre-requisites

- **COM S 3090** (software development practices)
- **COM S 2300** or **CPR E 3100** (Discrete Computational Structures)
- **ENGL 2500** (Written, Oral, Visual, and Electronic Composition)
- **Recommended:** *some familiarity with Unix*
 - We will use pyrite (cs server) and/or docker command line for some exercises

16

Administration

Professor: Myra Cohen

Office: Atanasoff 202

Office Hours: Monday 1:30-3:00
Thursday 3:30-4:30
Friday 2:00-3:00

Other times: by appointment

- email: mcohen@iastate.edu

**** please add COMS 4170 to email subject****

Web: <http://www.cs.iastate.edu/~mcohen>

17

Teaching Assistants

Faezeh Rajabi Kouchi

Email: faezehr@iastate.edu

Office Hours: Thursday 11:00 AM -1:00 PM

Location: Pearson 0112

Andrew De Gala

Email: ajdegala@iastate.edu

Office Hours: Wednesday 12:00-2:00 PM

Location: Pearson 0112

18

Communication

- We will utilize **discussion boards** on canvas for general questions regarding class content/assignments and schedules
- This will allow **all students** to see the answers
- I will utilize **canvas announcements** for general information

19

Assignments

- We will **have four** assignments that use testing tools and/or ask you to solve small problems related to what we have learned.
- These are meant to **enhance the theory** of this course.

20

Exercises

- There will be some in-class exercises and breakouts which are **meant to help with the assignments/exams**. These can be completed with groups in-class

21

Tests/Exams

- There are **2 exams** during the semester
 - March 3rd
 - April 26th
- The final exam period will be used for reflection and questions

22

Group Project

- There will be a group project/report where you learn about an advanced testing technique. **This is due on the Tuesday of prep week (May 5th)**. More details to follow.

23

Example Projects From Previous Years

- Automated **Program Repair**
- Fuzzing
- Regression **Test Selection**
- Test **Flakiness**
- Advanced **Combinatorial Testing**
- **Mobile App** Testing
- Using **LLMs to generate** test cases

24

Example Projects From Previous Years

- Advanced Mutation Testing
- User Acceptance Testing
- GUI (Vision Based) Testing
- **Test Generation** using Evolutionary Algorithms

25

Components for Grading

Assignments: **40%**

- Due: Feb 5th, Feb 26th, March 24th, April 21th

Two exams: **(20% each)**

- Dates: (Tues) March 3th, (Tues) April 28th

Group project report: **20%**

- Due: **Interim due date for teaming (mid-semester)**
- Due: **Tuesday, May 5th, (Note – this is during prep week)**
- **Final exam: As per the University Calendar (TB confirmed):**
 . >>
 • The final exam period will be used for students to meet with instructor and for course reflection
- **Exercises and assignments will have a late penalty. Details will be contained on the handouts.*

Extra Credit: There will be some extra credit – to be discussed

26

Reading Material



Book supplemental website (will use for some parts of course)
<https://www.albany.edu/faculty/offutt/softwaretest/>

27

Reading Material

- Most of the reading is from our textbook. When I indicate a chapter, it is from the textbook. **Some additional handouts will be added to Canvas.**
- Some lecture slides are derived from the textbook authors' material. *These are indicated with a copyright/acknowledgements.*
- We are using **Immediate Access** for the course textbook. Please read the details in the full syllabus. **Important: that you have 10 days to decline access!**

28

Course Delivery

- This course is **in person**
- I will try to record lectures and provide videos for **offline watching/reviewing** and I may **'flip'** some classes to allow better discussion during class. But they are **not meant to be a substitute for class**. I will also provide lecture notes.
- In class, I will **lecture**, present **tool demos** and have some **breakouts/exercises**
- I may present some lectures remotely due to weather (e.g. today), travel, or illness. For those Webex links will be made available in canvas

29

Elephant in the Room



30

Use of Generative AI

- Students may use gen AI tools to help prepare for assignments and projects. However, **all use must be attributed and a short reflection** on the quality of help provided by AI is required. Students are responsible for the accuracy of all generated content. Violation of this policy will result in zero on an assignment or project.
- Assignments and exercise material will appear on exams, hence you should **only rely on AI to help learn** rather than to achieve scores on assignments

31

Reading for this week

- Chapter 1 and 2 of our textbook

32

What Should We Test?



33

What Should We Test?



Unit tests are important, but there is more....

34

What is Testing



35

What is Testing



36

What is Testing



37

What is Testing



38

Some Definitions

- **Software fault:** A static defect in the software
- **Software failure:** External, incorrect behavior with respect to the requirements or other description of the expected behavior
- **Software error:** An incorrect internal state that is the manifestation of some fault

39

Example from Book

- Patient gives doctor list of symptoms (**failures**)
- Doctor tries to diagnose root cause or disease (**fault**)
- Doctor may look for internal irregularities (blood work, etc.) (**error**)

40

A Concrete Example

```

public static int numZero (int [] arr)
{ // Effects: If arr is null throw NullPointerException
  // else return the number of occurrences of 0 in arr
  int count = 0;
  for (int i = 1; i < arr.length; i++)
  {
    if (arr [ i ] == 0)
    {
      count++;
    }
  }
  return count;
}

```

Fault: Should start searching at 0, not 1

Test 1
[2, 7, 0]
Expected: 1
Actual: 1

Test 2
[0, 2, 7]
Expected: 1
Actual: 0

Error: i is 1, not 0, on the first iteration
Failure: none

Error: i is 1, not 0
Error propagates to the variable count
Failure: count is 0 at the return statement

41

Validation & Verification (IEEE)

Validation: The process of evaluating software at the end of software development to ensure compliance with intended usage

Verification: The process of determining whether the products of a given phase of the software development process **fulfill the requirements** established during the previous phase

IV&V- **independent** V&V

42

Testing Goals Based on Test Process Maturity

- Level 0 : There's no difference between **testing and debugging**
- Level 1 : The purpose of testing is to show **correctness**
- Level 2 : The purpose of testing is to show that the software **doesn't work**
- Level 3 : The purpose of testing is not to prove anything specific, but to **reduce the risk** of using the software
- Level 4 : Testing is a **mental discipline** that helps all IT professionals develop higher quality software

43

Level 0 Thinking

- Testing is the **same** as debugging
- Does not distinguish between incorrect **behavior** and mistakes in the program
- Does not help develop software that is **reliable** or **safe**

This is what we teach undergraduate CS majors

44

Level 1 Thinking

- Purpose is to show **correctness**
- Correctness **is impossible** to achieve
- What do we know if **no failures**?
 - Good software or bad tests?
- **Test engineers** have no:
 - Strict goal
 - Real stopping rule
 - Formal test technique
 - Test managers are **powerless**

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

45

45

Level 2 Thinking

- Purpose is to show **failures**
- Looking for failures is a **negative** activity
- Puts testers and developers into an **adversarial** relationship
- What if there are **no failures**?

This describes many software companies.
How can we move to a team approach ??

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

46

46

Level 3 Thinking

- Testing can only show the **presence of failures**
- Whenever we use software, we incur some **risk**
- Risk may be **small** and consequences unimportant
- Risk may be **great** and consequences catastrophic
- Testers and developers cooperate to **reduce risk**

This describes a few “enlightened” software companies

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

47

47

Level 4 Thinking

A mental discipline that increases quality

- Testing is only **one way** to increase quality
- Test engineers can become **technical leaders** of the project
- Primary responsibility to **measure and improve** software quality
- Their expertise should **help the developers**

This is the way “traditional” engineering works

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

48

48

A Testers Goal

- *To find faults and eliminate them as early as possible* in the testing process
- **NOT** to prove that software works
- This will:
 - Improve software quality
 - Improve customer's perceived experience
 - Reduce cost

49

For Thursday

- Start reading Chapter 1-2 in textbook

50

Week/Module	Topic/Concept	Course Materials	Activities/Assessments and Due Time
Wk. 1 (Jan 20, 22)	Introduction to Course	Required: Ch.1-2 in book	
Wk. 2 (Jan 27, 29)	Automated Testing intro and Exercise	Required: Ch. 3 in book	- In class exercise Jan 29 (can be completed in group or on own) - Files will be provided - bring laptops to class
Wk 3. (Feb 3, 5)	Test automation and Test First	Required: Ch. 4 in book	Assign one due Feb 5th
Wk 4. (Feb 10, 12)	Coverage and Test Automation	Required: Ch 5 in book	

51

Wk 5. (Feb 17, 19)	Automated Test Generation Tools in practice		In class exercise Tuesday Feb 24
Wk 6. (Feb 24, 26)	Automated Testing cont. Exam Review on Thursday		Assign 2 due Feb 26th
Wk 7. (Mar 3, 5)	Exam I Input Partitioning	Test Specification Paper Chapter 6.1	Exam I (Tuesday)
Wk 8. (Mar 10, 12)	Combination Strategies Combinatorial Testing	Read Chapter 6.2 -6.4	Team project handed out
Spring Break	March 17,19		

52




Week 9 (Mar 24,26)	Graph Coverage Project teaming	Read Chapter 7	• Assign 3 due
Week 10 (Mar 31, Apr 2)	Graphs Continued		
Week 11 (Apr 7,9)	Syntax and Mutation Testing	Chapter 9	In class exercise
Week 12 (Apr 14,16)	In class time to work on projects		

53




Week 13 (Apr 21, 23)	Metamorphic Testing Exam Review		Assign 4 due (Apr 21)
Week 14 (Apr 28,30)	Exam 2 Project time		Exam 2 Tuesday Apr 26th Extra credit assign due April 28th
Week 15 Prep week	Project discussions Course Wrap		Projects due Tuesday May 5th
Final Exam Period (TB confirmed)			Tuesday May 12, 9:45 AM

54